

ROADMAP — SISTEMA DE CINE

TP Programación IV · documento vivo

Documento vivo. Se actualiza sesión a sesión a medida que se completan fases o cambian decisiones. La fuente de verdad del análisis funcional (requisitos, casos de uso, modelo de datos) es externa a este documento; acá se traduce ese análisis en un plan de ejecución ordenado.

CONTEXTO

TP de Programación IV: una plataforma de cine completa (catálogo público, selección de butacas, checkout con Candy Bar/combos/cupones, fidelización, tickets QR, panel de administrador y panel de empleado para validación presencial).

El usuario pidió explícitamente:

- Avanzar **tarea por tarea**, no todo de una — cada entrega debe ser sólida y entendible antes de escalar a la siguiente.
- Un **diseño visual único**, no genérico: estética oscura/nocturna, cómoda a la vista, con motivos de cine (proyección, cinta de película, marquesina).
- El flujo debe soportar compra **anónima** y compra **registrada** (con beneficios/puntos) desde el catálogo público.
- Un panel de administrador con reportes y configuración integral.

DECISIONES CONFIRMADAS

DECISIÓN	ELEGIDO
Estilo visual	SCSS propio con design tokens (paleta oscura tipo sala de cine). Sin Angular Material ni Tailwind.
Pagos	Checkout con pasarela simulada (mock), persistida igual en la tabla <code>pagos</code> como aprobada.
Despliegue	Vercel
Componentes	Standalone por defecto. <code>compra</code> y <code>administracion</code> usan NgModule clásico (más componentes relacionados + valor pedagógico de la materia); el resto sigue standalone.
Manejo de estado	Signals de Angular + servicios inyectables. Sin NgRx.
Acceso a datos	<code>SupabaseService</code> central + servicios de dominio por feature. Nunca Supabase directo desde un componente.
Concurrencia de butacas	Validación de disponibilidad vía transacción/RPC en Postgres, no solo en el frontend.
Commits / PRs	Sin líneas de atribución al agente. Toda PR incluye Objetivo inicial / Qué se terminó haciendo / Resumen de cambios.
Idioma del código	Todo en español (componentes, servicios, rutas, variables) salvo API de Angular/TS/RxJS y vocabulario técnico de arquitectura (<code>core</code> , <code>shared</code> , <code>features</code> , <code>layout</code>).

Tests unitarios	No se generan salvo pedido explícito.
Archivos de componente	Siempre 3 separados (<code>.ts</code> / <code>.html</code> / <code>.css</code>), nunca template/estilos inline.

ESTRUCTURA DE CARPETAS

```
src/app/
  core/           # SupabaseService, guards, interceptors, modelos TS compartidos
  shared/         # UI primitives propias (botón, card, badge, spinner, modal...)
  layout/         # header, footer, shell con outlet
  features/
    catalogo/     # M02 - listado, detalle, búsqueda/filtro, reseñas
    salas-butacas/ # M03/M04 - mapa de butacas, selección, disponibilidad
    compra/       # M05/M06/M07 - carrito, candy/combos, cupones, pago mock (NgModule)
    fidelizacion/ # M08 - puntos, crédito, canjes
    entradas/     # M09 - ticket PDF + QR (vista cliente)
    cancelaciones/ # M10
    proximamente/ # M11 - estrenos, alertas
    perfil/       # M01 - registro, login, datos del cliente, Mis películas
    empleado/     # validación QR/código manual
    administracion/ # M12/M13 + ABM de todo lo anterior (NgModule)
  app.routes.ts   # composición de rutas lazy por feature
```

ROADMAP POR FASES

Leyenda de estado:  **Hecho** ·  **En progreso** ·  **Pendiente**

Fase 0 — Fundaciones del proyecto

#	SUB-TAREA	ESTADO
0.1	Limpieza de repo y documentación base (README + CHANGELOG)	
0.2	Estructura de carpetas (<code>features/*</code> + rutas lazy placeholder)	
0.3	Supabase: cliente, <code>environment.ts</code> , <code>SupabaseService</code>	
0.4	Design system base: variables SCSS + componentes <code>Button</code> / <code>Card</code>	
0.5	Shell de layout: header, footer, fondo temático	
0.6	Esquema SQL inicial en Supabase (tablas base + RLS)	
0.7	Seed de datos de demo	

Verificación 0.2 (hecha): `ng build` genera un chunk lazy por feature, `ng serve` levanta la app y navega entre las 10 páginas placeholder vía la navegación provisoria. `core/` / `shared/` / `layout/` se crean recién en 0.3/0.4/0.5, cuando tengan contenido real (git no versiona carpetas vacías).

Fase 1 — Catálogo público (M02)

Listado de películas (poster, nombre, duración, clasificación), detalle (sinopsis, géneros, reseñas con estrellas y promedio, funciones disponibles), buscador + filtro múltiple por género, destacado "3 más vendidas" en home.

#	SUB-TAREA	ESTADO
1.1	Capa de datos: modelos + <code>PeliculasService</code> (listado/detalle/destacadas) + RPC agregada de "más vendidas"	☒
1.2	Página de listado (home): grilla de películas + destacado "3 más vendidas"	☐
1.3	Buscador + filtro múltiple por género	☐
1.4	Página de detalle de película (sinopsis, géneros, funciones disponibles, reseñas + promedio)	☐

Fase 2 — Salas, butacas y funciones (M03/M04)

Mapa visual de butacas (normal/accesible/VIP con diferenciación de color clara), disponibilidad en tiempo real, selección de función desde el detalle.

Fase 3 — Autenticación y perfil (M01)

Registro (mail, nombre, apellido, fecha nacimiento, tipo de sangre, color de ojos, vacaciones anuales), login/logout, perfil (datos, puntos, crédito, historiales) — sin romper la navegación de invitado.

Fase 4 — Compra: entradas + Candy/combos + cupones (M05/M06/M07)

Carrito (entradas + productos + combos), cupón, compra anónima vs registrada, validación de edad (RNo4), pantalla de pago simulada, persistencia de venta.

Fase 5 — Entradas y QR (M09)

PDF de entrada + QR, pantalla "Mis entradas".

Fase 6 — Panel de empleado: validación (M09 operación)

Escaneo/ingreso manual de código, validación de entrada y retiro de Candy, invalidación de QR usado.

Fase 7 — Panel de administración: ABM base (M03/M04/M06/M07 admin)

CRUD de películas, salas/butacas, funciones (asignación automática de sala + RNo1/RNo2/RNo3), productos/categorías, combos, cupones y preventa.

Fase 8 — Fidelización y crédito (M08)

Acreditación de puntos, configuración y canje de recompensas, saldo e historial, no transferencia.

Fase 9 — Cancelaciones (M10)

Cancelación hasta 2 horas antes, liberación de butacas, generación de crédito.

Fase 10 — Próximamente y notificaciones (M11)

Sección "Próximamente", alertas de estreno, preventa 7 días antes, notificación in-app al habilitarse la venta.

Fase 11 — Reportes y estadísticas (M12)

Facturación diaria, entradas vendidas, exportación PDF/Excel, gráficos de películas más vistas y producto más vendido.

Fase 12 — Auditoría (M13) ■

Registro y consulta de logs de acciones administrativas y de validación.

Fase 13 — PWA y pulido final ■

Manifest + service worker, instalabilidad, revisión de RLS, accesibilidad, responsive final, despliegue a Vercel, README de arquitectura (RNF-008).

PUNTOS ABIERTOS (SE RESUELVEN AL LLEGAR A LA FASE CORRESPONDIENTE)

- ~~Edad en compra anónima (Fase 4)~~ — **resuelto**: el checkout anónimo pide fecha de nacimiento y se valida igual que con un perfil registrado (ver `docs/01_Analisis_Funcional_Cine.pdf`, sección 11, y `docs/03_Modelo_de_Datos_Supabase_Cine.pdf`, tabla `ventas`, columna `fecha_nacimiento_comprador`).
- **Canal de notificaciones de estreno** (Fase 10): arrancar in-app, evaluar email después.
- **Borrado físico vs soft-delete** (Fase 7): se propone soft-delete (`activo` / `activa`) salvo indicación contraria.
- **Layout gráfico de butacas accesibles** (Fase 2): se define con un mockup simple antes de codear.

CONVENCIÓN TRANSVERSAL

- Cada PR agrega una entrada nueva en `CHANGELOG.md`.
- El `README.md` se actualiza cuando una fase introduce una decisión de arquitectura nueva.
- Cada PR describe: Objetivo inicial / Qué se terminó haciendo / Resumen de cambios. Sin líneas de atribución al agente.